

Laborator 6

Modelarea structurală

1 Modelarea structurală

Modelarea structurală în Verilog se referă la descrierea unui circuit logic folosind module interconectate. Spre deosebire de modelarea comportamentală care se axează pe descrierea comportamentului unui circuit, modelarea structurală se axează pe descrierea interconectării diferitelor module ce împreună îndeplinesc o funcție logică. Modelarea structurală permite o reprezentare ierarhică a circuitelor, abstractizând complexitatea acestora (circuite complexe pot fi realizate interconectând multiple circuite mai simple).

Printre avantajele modelării structurale se numără modularitatea, posibilitatea de a refolosi module și ușurința prin care poate fi înțeles un circuit complex atunci când acesta este împărțit în module de complexitate mai mică. Modelarea structurală are însă limitări în ceea ce privește descrierea comportamentului complex al unui modul, axându-se pe interconectivitate.

2 Aspectele modelării structurale

În modelarea structurală, modulele sunt folosite pentru a încapsula o funcționalitate specifică a unei componente dintr-un circuit digital. Modulele servesc ca elemente constitutive ale unui circuit digital ce pot fi instanțiate și conectate pentru a crea un sistem mai mare.

Instanțele reprezintă copii sau replici ale unui modul ce sunt create în ierarhia de proiectare.

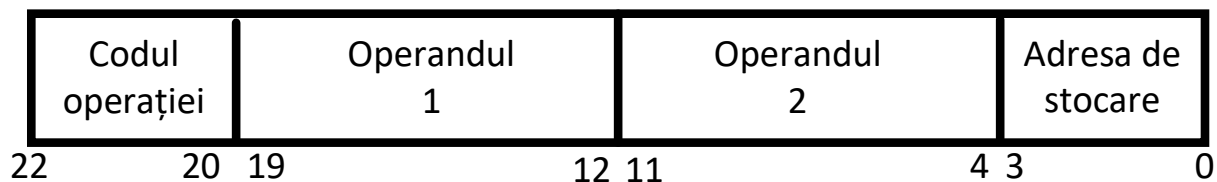
Conexiunile se referă la legăturile dintre diversele module din ierarhia proiectată și sunt utilizate pentru a transfera date și semnale de control.

Ierarhia se referă la organizarea modulelor într-un model structural. Aceasta permite o abordare de proiectare „top-down” în care un modul complex este descompus în module mai simple și mai ușor de gestionat.

3 Exerciții

Folosind modulele dezvoltate în laboratoarele anterioare, implementați un procesor. Funcționalitatea acestuia este descrisă mai jos.

În tabelul de mai jos este descris formatul unei instrucțiuni pentru procesorul ce trebuie implementat, precum și descrierea instrucțiunilor disponibile:



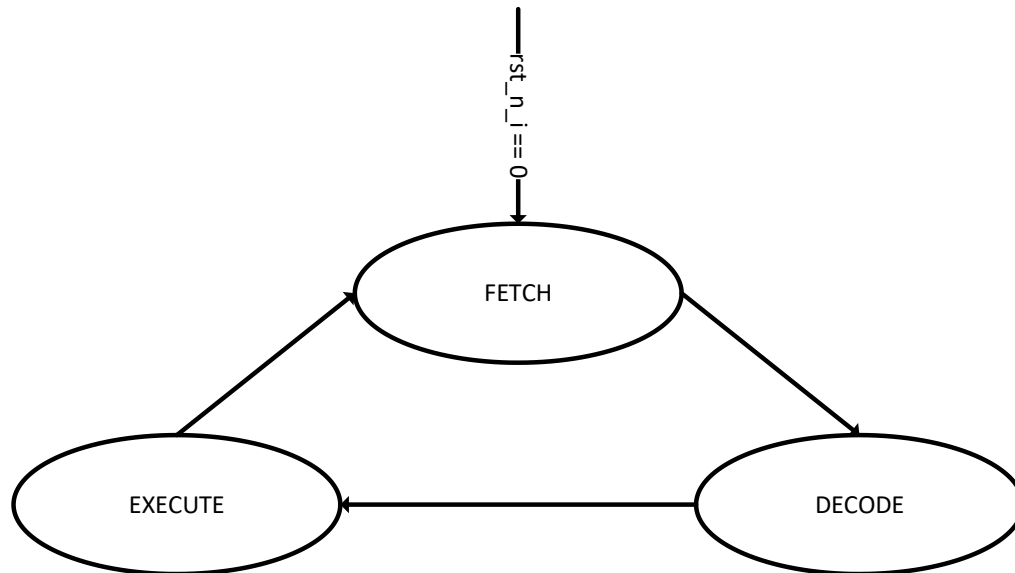
Numele câmpului	Poziția	Numărul de biți	Explicații
Codul operației	22-20	3	Codul operației ce va fi executată.
Operandul 1	19-12	8	Primul operand al operației ce va fi executată.
Operandul 2	11-4	8	Al doilea operand al operației ce va fi executată.
Adresa de stocare	3-0	4	Adresa unde va fi stocat rezultatul operației

Numele operației	Codul operației în format binar	Exemplu de utilizare	Explicații
AND	000	AND op1 op2 a0	„ȘI” între op1 și op2. Rezultatul este stocat la adresa a0.
ORR	001	ORR op1 op2 a1	„SAU” între op1 și op2. Rezultatul este stocat la adresa a1.
XOR	010	XOR op1 op2 a2	„SAU EXCLUSIV” între op1 și op2. Rezultatul este stocat la adresa a2.
MOV	011	MOV op1 x a3	Încarcă adresa a3 cu op1. Valoarea operandului 2 nu este luată în considerare.
ADD	100	ADD op1 op2 a15	Adunare între op1 și op2. Rezultatul este stocat la adresa a15.
SUB	101	SUB op1 op2 a11	Scădere între op1 și op2. Rezultatul este stocat la adresa a11.
SHL	110	SHL op1 op2 a10	Deplasare la stânga a op1 cu op2 biți. Rezultatul este stocat la adresa a10.
SHR	111	SHR op1 op2 a8	Deplasare la dreapta a op1 cu op2 biți. Rezultatul este stocat la adresa a8.

Modulele ce alcătuiesc procesorul sunt descrise în cele ce urmează (pe baza schemei atașate în laborator: doc/cpu.jpeg):

- Modulul de control (control_unit)

Acesta reprezintă un automat de stare ce are rolul de a gestiona celelalte componente ale procesorului. Graful de tranziție a stărilor este prezentat în figura de mai jos:



La resetare, procesorul intră în starea FETCH. În această stare, o instrucțiune din memoria de instrucțiuni este citită de modulul de control. Următoarea stare poartă numele DECODE. În această stare se decodează instrucțiunea citită anterior, determinând ce operație trebuie efectuată, ce operații trebuie utilizate și la ce adresă se va stoca rezultatul. Din starea FETCH se trece în starea EXECUTE. În această stare, parametrii determinați anterior sunt transmiși către unitatea de calcul (unitatea aritmetică și logică/arithmetic logic unit – ALU) împreună cu semnalele de control pentru memoria de date, urmând ca mai apoi ciclul să fie reluat.

- Unitatea aritmetică și logică (ALU)

Acest modul are rolul de a implementa efectiv operațiile procesorului și de a le efectua pe baza unei selecții venite de la modulul de control.

- Program counter

Acest modul are rolul de a calcula adresa instrucțiunilor ce urmează a fi efectuate.

- Memoria de instrucțiuni

În cadrul acestui laborator, memoria de instrucțiuni este configurată ca „read-only”. Aceasta are rolul de a stoca instrucțiunile programului ce urmează a fi rulat.

- Memoria de date

Acest modul are rolul de a stoca rezultatele instrucțiunilor executate.

Pași pentru implementarea procesorului:

1. Conectați în interiorul modulului cpu restul modulelor puse la dispoziție în laborator, folosind schema din directorul doc (cpu.jpeg).
2. Examinați programul program.clp din directorul scripts.
3. Din linia de comandă apăsați script-ul clp_compiler.py folosind comanda „python .\clp_compiler.py” din directorul scripts. Acesta va crea pe baza programului program.clp un fișier binar (sim/program.memb).
4. Examinați codul Verilog pentru testbench.
5. Rulați o simulare și examinați funcționarea procesorului.